Performance Analysis of Homomorphic Encryption in Neural Networks*

Feng Hao, G2504051D and Wei Xiao, G2405459F

Abstract. The rapid adoption of AI-as-a-Service (AIaaS) introduces significant data privacy risks when transmitting sensitive user data to untrusted third-party cloud servers. Homomorphic Encryption (HE) offers a mathematically rigorous solution by allowing computations directly on encrypted data without decryption. In this application-based project, we evaluate the performance overhead and approximation errors of deploying HE in neural network inference. We adapt a standard image classification model into an HE-friendly shallow polynomial network and implement the CKKS encryption scheme using the TenSEAL library. Our evaluation on the MNIST dataset demonstrates that while HE successfully maintains 100% classification accuracy with a negligible approximation error of 0.031, it introduces severe performance bottlenecks, increasing the average inference latency from 0.50 milliseconds to 2.94 seconds per image (≈6,000× overhead). We further analyse the full end-to-end serialization pipeline and find that transmitting the public CKKS context alone requires 117 MB of bandwidth—a significant communication cost driven by the Galois key set. These results highlight the practical trade-offs between absolute data confidentiality and computational efficiency in modern privacy-preserving AI systems.

Keywords: Homomorphic Encryption · Privacy-Preserving AI · Machine Learning · TenSEAL · CKKS.

1 Introduction

1.1 Motivation

The rapid proliferation of cloud computing and Artificial Intelligence-as-a-Service (AIaaS) has transformed how organizations deploy and consume machine learning models. In this paradigm, users transmit sensitive data—ranging from personal medical records to proprietary financial information—to third-party cloud servers for high-performance inference. However, this architecture introduces a critical privacy bottleneck: traditional encryption only protects data in transit and at rest. To be processed by an AI model, data must typically be decrypted, exposing the plaintext to potentially compromised cloud environments or 'honest-but-curious' service providers.

* The source code for this project is available at: `https://github.com/CMKSJ/Privacy-Preserving`

To bridge the gap between utility and confidentiality, there is an urgent need for techniques that enable computation directly on encrypted data. **Homomorphic Encryption (HE)** has emerged as a mathematically rigorous solution to this challenge, enabling 'blind computation' where a server can perform mathematical operations on ciphertexts without ever accessing the underlying plaintext. Despite its theoretical promise, the practical adoption of HE is severely hindered by the massive computational overhead and latency it introduces compared to plaintext operations. Understanding these trade-offs is essential for designing viable privacy-preserving AI systems.

1.2 Research Question

This project aims to answer a fundamental question: *In a realistic AI-as-a-Service architecture, what is the quantitative computational overhead and accuracy trade-off of applying Homomorphic Encryption (HE) to achieve zero-knowledge cloud inference?*

Specifically, we investigate whether a shallow neural network can maintain its predictive utility while operating under the performance constraints of the CKKS encryption scheme.

1.3 Contributions

To address the research question, we implemented a privacy-preserving pipeline and performed a comprehensive evaluation:

- We designed an HE-friendly neural network by replacing non-linear activation functions (e.g., ReLU) with a second-order polynomial function (x^2), ensuring compatibility with homomorphic operations.
- We implemented an end-to-end secure inference system using the TenSEAL library, deploying the CKKS scheme to protect user data throughout the inference lifecycle.
- We performed a rigorous benchmarking on the MNIST dataset, quantifying the impact of HE on classification accuracy, floating-point approximation errors, and end-to-end latency.
- We implemented and evaluated a full serialization-based client-server pipeline, measuring per-phase latency and data transmission costs including the 117 MB public context overhead.

2 Background

Homomorphic Encryption (HE) is a cryptographic scheme that allows arbitrary mathematical operations to be performed on ciphertexts. When the resulting ciphertext is decrypted, it yields the exact result of the operations as if they had been performed on the plaintext. In this project, we utilize the **CKKS (Cheon-Kim-Kim-Song)** scheme, which is specifically designed to support approximate arithmetic over arrays of complex and real numbers, making it uniquely suitable for neural network floating-point operations.

3 Methodology and System Design

3.1 System Architecture

Our implementation separates the workflow into a Client environment and an untrusted Cloud Server environment. The Client generates the cryptographic keys, encrypts the raw image into a ciphertext vector, and transmits it. The Cloud Server, possessing only the encrypted input and a plaintext pre-trained model, performs matrix multiplications and additions blindly. The encrypted result is sent back to the Client for local decryption.

3.2 HE-Friendly Model Adaptation

Standard neural networks rely heavily on non-linear activation functions such as ReLU ($max(0, x)$). However, condition-based operations are prohibitively expensive or unsupported in HE. To solve this, we constructed a shallow neural network consisting of two fully connected layers ($784 \rightarrow 64 \rightarrow 10$) and utilized a **polynomial activation function** ($f(x) = x^2$). Since HE inherently supports homomorphic additions and multiplications, this squared activation perfectly integrates into the encrypted computation pipeline.

3.3 Cryptographic Parameter Selection

In the CKKS scheme, each multiplication operation consumes the scaling factor, requiring rescaling which subsequently consumes the polynomial modulus depth. To support our network's required 3-level multiplicative depth without triggering a `scale out of bounds` error, we configured the TenSEAL context with the following parameters:

- **Polynomial Modulus Degree:** Set to 16384 to provide sufficient capacity for the prime number chain.
- **Coefficient Modulus Bit Sizes:** Set to `[60, 40, 40, 40, 60]`, providing exactly *three* intermediate 40-bit primes that match our 3-step pipeline depth.
- **Global Scale:** Set to 2^{40}, matching the 40-bit intermediate primes to maximise numerical precision while avoiding overflow during rescaling.
- **Galois Keys:** Generated to support ciphertext rotation, which is required internally during the `matmul` operation.
- **Relinearization Keys:** Generated to reduce ciphertext size back to two components after each ciphertext-ciphertext multiplication (i.e., the `square` activation).

3.4 Encrypted Inference Pipeline

The server-side inference consists of exactly three homomorphic operations that map to the two-layer network:

1. `enc_out1 = enc_x.matmul(w1) + b1` — Encrypted fully-connected layer 1, consuming one depth level; auto-rescaling is triggered to maintain scale alignment.
2. `enc_sq = enc_out1.square()` — Polynomial activation x^2, consuming one depth level; followed by automatic relinearization and rescaling.
3. `enc_out2 = enc_sq.matmul(w2) + b2` — Encrypted fully-connected layer 2, producing 10 encrypted logits and consuming the final depth level.

This three-step structure precisely exhausts the three intermediate primes in `[60, 40, 40, 40, 60]`, leaving the ciphertext in a valid decryptable state with no wasted depth budget.

4 Experimental Evaluation

4.1 Experimental Setup

The experiments were conducted using the MNIST handwritten digit dataset (60,000 training / 10,000 test images, 28×28 grayscale). The polynomial network was trained with the Adam optimizer ($lr = 0.001$, batch size 64) using cross-entropy loss for 3 epochs, yielding a model with 50,890 parameters. All experiments were executed in a reproducible Docker environment (`python:3.11-slim` with PyTorch 2.10.0 and TenSEAL built from source) on a CPU-only host. The inference weights were extracted and applied directly to the TenSEAL encrypted operations. We ran a batch evaluation of 100 test images to compare the plaintext baseline against the HE inference, and a separate serialization-based pipeline experiment to measure full end-to-end communication overhead.

4.2 Accuracy and Approximation Error

One of the most critical findings is that HE inference maintained a **100% match** with the plaintext model's predictions. Because the CKKS scheme is an approximate computation method, it introduces noise to the floating-point values. As shown in Table 1, the HE computed logits differ from the plaintext logits by an average absolute error of 0.0312.

However, since image classification relies on the `Argmax` function (the relative maximum value among the 10 classes) rather than absolute values, this minor error did not alter the final prediction outcome.

As illustrated in Table 1, the maximum absolute error across all classes remains within the range of 10^{-2}. Specifically, for the target class '7', the ciphertext logit (24.858727) is slightly lower than the plaintext logit (24.877792), yet it significantly dominates all other potential classes. This experimental evidence confirms that the floating-point approximation noise inherent in the CKKS scheme does not compromise the classification integrity for shallow polynomial networks.

Table 1: Detailed Logits Comparison and Absolute Error (Sample Label: 7)

Class	Plaintext Logits	HE Ciphertext Logits	Absolute Error
0	4.482852	4.482895	0.00004362
1	2.121562	2.113931	0.00763056
2	4.205320	4.205511	0.00019015
3	8.347394	8.350828	0.00343423
4	-9.862619	-9.873343	0.01072356
5	0.092624	0.085282	0.00734170
6	-9.865355	-9.848685	0.01667064
7 (Pred)	**24.877792**	**24.858727**	**0.01906498**
8	-2.058335	-2.055990	0.00234520
9	8.068798	8.042974	0.02582402

4.3 Performance Overhead Analysis

While accuracy was perfectly preserved, the computational cost was severe. In our batch evaluation, the plaintext inference required an average of **0.50 ms** per image. In contrast, the HE inference required **2.94 seconds** per image — an increase in latency by a factor of approximately **6,000**×. This massive overhead is attributed to the complex polynomial arithmetic of ciphertext matrix multiplications and the necessity of continuous *relinearization* and *rescaling* after each layer.

4.4 Serialization and Communication Overhead

To simulate a realistic client-server deployment, we implemented a full serialization pipeline where the client and server exchange only serialized byte streams (no shared in-memory context). Table 2 summarises the end-to-end latency breakdown and the sizes of all exchanged objects for a single inference.

Table 2: End-to-end latency and communication cost (single image, label = 7)

Phase	Time (s)	Data Size
Client: key generation + encryption	1.156	—
Public context (Galois + Relin keys)	—	117.0 MB
Input ciphertext (784-dim)	—	835.3 KB
Server: deserialize + infer	3.090	—
Encrypted result (10-dim)	—	256.1 KB
Client: decryption	<0.001	—
Total end-to-end	**4.247**	**≈118 MB**

A notable finding is the disproportionate size of the public context (≈117 MB), which dominates the communication cost. This is caused by the Galois key set, required to enable ciphertext rotation during the `matmul` operation. While the Galois keys only need to be transmitted once per session, their size represents a significant bandwidth requirement for low-latency applications. By contrast, the actual input and output ciphertexts are comparatively compact (835 KB and 256 KB respectively).

5 Discussion and Future Work

Our results validate that Homomorphic Encryption can provide absolute data privacy for AIaaS without compromising accuracy, provided the network is shallow. However, the $O(10^3)$ performance penalty makes current pure HE solutions impractical for real-time, latency-sensitive applications. Furthermore, because approximation errors accumulate multiplicatively, applying CKKS to very deep networks (e.g., ResNet) would eventually cause the noise to eclipse the actual data, breaking the `Argmax` outcome.

Future work should explore hybrid approaches — such as combining Multi-Party Computation (MPC) with HE — or utilizing hardware acceleration (e.g., FPGAs) to drastically reduce the matrix multiplication latency in the encrypted domain.

6 Conclusion

This project successfully demonstrates the implementation and benchmarking of a privacy-preserving image classification system. We conclude that while Homomorphic Encryption elegantly solves the trust and privacy issues inherent in cloud AI, its extreme computational overhead currently restricts its viability to offline, asynchronous batch-processing tasks where security strictly outweighs speed.

References

1. Fauzi, P., Zhang, J.: Privacy-Preserving Technologies & Security in AI. Nanyang Technological University Course Material (2026).
2. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic Encryption for Arithmetic of Approximate Numbers. Advances in Cryptology - ASIACRYPT 2017, pp. 409–437. Springer (2017).
3. Benaissa, A., et al.: TenSEAL: A library for doing homomorphic encryption operations on tensors. `https://github.com/OpenMined/TenSEAL`.